Documentation

STEP 1

Code:

```
import pandas as pd
import numpy as np
import sqlite3
import requests

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.impute import SimpleImputer
```

Interpretation:

This cell imports all the libraries required for the project. Pandas and NumPy are used for data handling and numerical operations, sqlite3 lets the notebook talk to a SQL database, requests is used to call a web API, Matplotlib and Seaborn are used for plotting, and SimpleImputer from scikit-learn is imported for handling missing values later in the pipeline.

STEP 2

Code:

```
customers = pd.read_csv("/content/customers - customers.csv.csv")

customers.head()
```

Interpretation:

This cell loads the customer master data from a CSV file into a DataFrame called customers and displays the first five rows. This gives a first look at customer-level fields such as ID, name, age, gender and city before any processing begins.

STEP 3

Code:

```
transactions = pd.read_json("/content/transactions.json")

transactions.head()
```

Interpretation:

This cell reads transaction records stored in a JSON file into a DataFrame called transactions and previews the first five rows. This is the second data source that will later be combined with customer information.

STEP 4

Code:

```
with open('/content/products (1).sql', 'r') as f:
    sql_content = f.read()
print(sql_content)
```

Interpretation:

This cell opens a SQL file containing product information and reads its raw text content so that it can be executed against a database in the next step.

STEP 5

Code:

```
conn = sqlite3.connect(':memory:') # Create an in-memory SQLite database
conn.executescript(sql_content) # Execute the SQL script to create and populate the table

products = pd.read_sql("SELECT * FROM products", conn)

products.head()

conn.close()
```

Interpretation:

This cell creates a temporary in-memory SQLite database, runs the SQL script to build and populate the products table, then loads that table into a DataFrame called products using a SQL query. The connection is closed afterwards to free resources. This is the third data source in the pipeline.

STEP 6

Code:

```
url = "https://dummyjson.com/users"

response = requests.get(url)

api_data = response.json()

api_df = pd.DataFrame(api_data["users"])

api_df.head()
```

Interpretation:

This cell calls a public REST API (dummyjson.com) to fetch additional user records, converts the JSON response into a DataFrame called api_df, and previews it. This is the fourth and final data source, used later to enrich the customer profile with extra demographic fields.

STEP 7

Code:

```
customer_transaction = pd.merge(
    customers,
    transactions,
    on="customer_id",
    how="left"
)

customer_transaction.head()
```

Interpretation:

This cell merges the customers table with the transactions table on the shared customer_id column using a left join, so every customer keeps their row even if a match isn't found, and stores the result as customer_transaction.

STEP 8

Code:

```
merged_data = pd.merge(
    customer_transaction,
    products,
    on="product_id",
    how="left"
)

merged_data.head()
```

Interpretation:

This cell merges the combined customer-transaction data with the products table on product_id using a left join, attaching product details (such as category and price) to every transaction row.

STEP 9

Code:

```
merged_data = pd.merge(
    customer_transaction,
    products,
    on="product_id",
    how="left"
)

merged_data.head()
```

Interpretation:

This cell repeats the same merge between customer_transaction and products on product_id. It produces the same merged_data as the previous step; keeping it here effectively re-runs the join, for example after upstream data changes.

STEP 10

Code:

```
final_data = pd.merge(
    merged_data,
    api_df,
    left_on="customer_id",
    right_on="id",
    how="left"
)

final_data.head()
```

Interpretation:

This cell brings in the fourth data source by merging merged_data with the API-based api_df, matching the dataset's customer_id to the API's id field. The result, final_data, is now the single unified dataset that all later steps work from.

STEP 11

Code:

```
print("Rows and Columns:", final_data.shape)
```

Interpretation:

This cell prints the number of rows and columns in final_data, giving a quick sense of the overall size of the merged dataset.

STEP 12

Code:

```
final_data.head()
```

Interpretation:

This cell displays the first five rows of final_data to visually confirm that the four sources merged correctly and that the combined columns look as expected.

STEP 13

Code:

```
final_data.tail()
```

Interpretation:

This cell displays the last five rows of final_data, which helps check the end of the dataset for any irregularities such as trailing blank or mismatched rows.

STEP 14

Code:

```
final_data.info()
```

Interpretation:

This cell runs info() on final_data to show the data type of every column, how many non-null values each column contains, and the overall memory footprint of the DataFrame.

STEP 15

Code:

```
final_data.describe(include="all")
```

Interpretation:

This cell runs describe(include="all") to generate summary statistics for every column, both numerical (mean, standard deviation, min/max, quartiles) and categorical (count, unique values, most frequent value).

STEP 16

Code:

```
final_data.isnull().sum()
```

Interpretation:

This cell counts the missing (null) values in each column of final_data, which is the first step in understanding how much cleaning will be needed before modelling.

STEP 17

Code:

```
final_data.duplicated().sum()
```

Interpretation:

This cell counts the number of fully duplicated rows in final_data so that repeated records can be identified before they distort any analysis.

STEP 18

Code:

```
final_data.dtypes
```

Interpretation:

This cell lists the data type assigned to every column, which helps confirm whether numeric-looking columns are stored as numbers and dates are stored as text (and therefore need conversion later).

STEP 19

Code:

```
final_data.columns
```

Interpretation:

This cell prints the full list of column names in final_data, giving a complete reference of the fields available for analysis and modelling.

STEP 20

Code:

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.style.use("ggplot")
```

Interpretation:

This cell re-imports the plotting libraries and switches Matplotlib to the 'ggplot' visual style, giving all charts that follow a consistent, cleaner look.

STEP 21

Code:

```
numerical_columns = final_data.select_dtypes(include=['int64','float64']).columns

print(numerical_columns)
```

Interpretation:

This cell selects only the numeric (int64/float64) columns from final_data and prints their names, isolating the fields that will be used in numerical analysis such as histograms and correlation.

STEP 22

Code:

```
categorical_columns = final_data.select_dtypes(include=['object']).columns

print(categorical_columns)
```

Interpretation:

This cell selects only the text/object columns from final_data and prints their names, isolating the categorical fields that will later need encoding.

STEP 23

Code:

```
import matplotlib.pyplot as plt

numerical_columns = final_data.select_dtypes(include=["int64","float64"]).columns

final_data[numerical_columns].hist(
    figsize=(15,10),
```

```
    bins=20,
    color="skyblue",
    edgecolor="black"
)

plt.suptitle("Distribution of Numerical Features", fontsize=16)

plt.show()
```

Interpretation:

This cell plots histograms for every numerical column in a single grid of subplots, showing how the values in each numeric feature are distributed (e.g. skewed, normal, or clustered).

STEP 24

Code:

```
for column in numerical_columns:
    # Only plot if the column has at least one non-null value
    if final_data[column].notna().any():
        plt.figure(figsize=(6,4))

        sns.boxplot(
            x=final_data[column]
        )

        plt.title(f"Boxplot of {column}")

        plt.show()
    else:
        print(f"Skipping boxplot for '{column}' as it contains only NaN values.")
```

Interpretation:

This cell loops through every numerical column and draws an individual boxplot for it, skipping any column that contains only missing values. Boxplots make it easy to spot outliers and understand the spread of each feature one at a time.

STEP 25

Code:

```
print(final_data[numerical_columns].skew())
```

Interpretation:

This cell calculates and prints the skewness of each numerical column, a statistic that indicates whether a feature's distribution leans to the left or right of a normal, symmetric shape.

STEP 26

Code:

```
print(final_data.columns)
```

Interpretation:

This cell reprints the column names of final_data, used here as a quick checkpoint before the correlation and visualization steps that follow.

STEP 27

Code:

```
correlation = final_data.corr(numeric_only=True)

print(correlation["amount"].sort_values(ascending=False))
```

Interpretation:

This cell computes the correlation matrix for all numeric columns and prints how strongly each feature correlates with the transaction amount column, sorted from strongest positive to strongest negative relationship.

STEP 28

Code:

```
categorical_columns = final_data.select_dtypes(include=["object"]).columns

for col in categorical_columns:

    plt.figure(figsize=(8,4))

    sns.countplot(
        data=final_data,
        x=col,
        palette="viridis"
    )

    plt.xticks(rotation=45)

    plt.title(f"Countplot of {col}")

    plt.show()
```

Interpretation:

This cell loops through every categorical column and draws a countplot for it, showing how many records fall into each category (for example, how many transactions used each payment mode).

STEP 29

Code:

```
plt.figure(figsize=(8,5))

sns.scatterplot(
    data=final_data,
    x="age_x",
```

```
    y="amount",
    color="red"
)

plt.title("Age vs Transaction Amount")

plt.show()
```

Interpretation:

This cell creates a scatter plot of customer age against transaction amount, helping to visually check whether older or younger customers tend to spend more or less.

STEP 30

Code:

```
plt.figure(figsize=(8,5))

sns.boxplot(
    data=final_data,
    x="payment_mode",
    y="amount",
    palette="Set2"
)

plt.xticks(rotation=20)

plt.title("Payment Mode vs Transaction Amount")

plt.show()
```

Interpretation:

This cell creates a boxplot comparing transaction amount across different payment modes, showing how the spread and typical spending differs depending on how a customer paid.

STEP 31

Code:

```
plt.figure(figsize=(10,8))

corr = final_data[numerical_columns].corr()

sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm",
    linewidths=0.5
)

plt.title("Correlation Heatmap")

plt.show()
```

Interpretation:

This cell builds a correlation heatmap for all numerical columns, using colour and printed correlation values to make it easy to spot which features move together.

STEP 32

Code:

```
final_data.groupby("payment_mode")["amount"].mean()
```

Interpretation:

This cell groups the data by payment_mode and calculates the average transaction amount for each group, summarising spending behaviour by payment method in a simple table.

STEP 33

Code:

```
plt.figure(figsize=(8,5))

sns.violinplot(
    data=final_data,
    x="payment_mode",
    y="amount",
    palette="pastel"
)

plt.title("Transaction Amount by Payment Mode")

plt.show()
```

Interpretation:

This cell draws a violin plot of transaction amount by payment mode, which shows both the spread and the shape (density) of the amount distribution for each payment type in one chart.

STEP 34

Code:

```
# Check missing values

final_data.isnull().sum()
```

Interpretation:

This cell re-checks the number of missing values in each column of final_data, confirming the starting point right before the different missing-value handling techniques are applied.

STEP 35

Code:

```
numerical_cols = final_data.select_dtypes(include=["int64","float64"]).columns

categorical_cols = final_data.select_dtypes(include=["object"]).columns

print("Numerical Columns:")
print(numerical_cols)

print("\nCategorical Columns:")
print(categorical_cols)
```

Interpretation:

This cell separates the column names into two groups — numerical and categorical — and prints both lists, setting up which imputation technique will be applied to which type of column.

STEP 36

Code:

```
from sklearn.impute import SimpleImputer

simple_mean = final_data.copy()

mean_imputer = SimpleImputer(strategy="mean")

# Identify numerical columns that are not entirely NaN
imputable_numerical_cols = numerical_cols[simple_mean[numerical_cols].notna().any()]

simple_mean[imputable_numerical_cols] = mean_imputer.fit_transform(
    simple_mean[imputable_numerical_cols]
)

simple_mean.isnull().sum()
```

Interpretation:

This cell makes a copy of the dataset and uses scikit-learn's SimpleImputer with a 'mean' strategy to fill missing values in the numerical columns that have at least one valid value, then checks how many nulls remain afterwards.

STEP 37

Code:

```
simple_median = final_data.copy()

median_imputer = SimpleImputer(strategy="median")

# Identify numerical columns that are not entirely NaN
imputable_numerical_cols = numerical_cols[simple_median[numerical_cols].notna().any()]

simple_median[imputable_numerical_cols] = median_imputer.fit_transform(
    simple_median[imputable_numerical_cols]
)
```

```
simple_median.isnull().sum()
```

Interpretation:

This cell repeats the same idea using a 'median' strategy instead of the mean, filling missing numerical values with each column's median and then re-checking for remaining nulls. Median imputation is less sensitive to extreme values than the mean.

STEP 38

Code:

```
mode_data = final_data.copy()

mode_imputer = SimpleImputer(strategy="most_frequent")

# Identify categorical columns that are not entirely NaN
imputable_categorical_cols = categorical_cols[mode_data[categorical_cols].notna().any()]

mode_data[imputable_categorical_cols] = mode_imputer.fit_transform(
    mode_data[imputable_categorical_cols]
)

mode_data.isnull().sum()
```

Interpretation:

This cell applies SimpleImputer with a 'most_frequent' strategy to the categorical columns, filling missing text values with the most commonly occurring category in each column.

STEP 39

Code:

```
constant_data = final_data.copy()

constant_imputer = SimpleImputer(
    strategy="constant",
    fill_value="Unknown"
)

constant_data[categorical_cols] = constant_imputer.fit_transform(
    constant_data[categorical_cols]
)

constant_data.isnull().sum()
```

Interpretation:

This cell uses a 'constant' strategy to fill every missing value in the categorical columns with the placeholder text 'Unknown', which keeps missing information visible as its own category rather than guessing a value.

STEP 40

Code:

```
from sklearn.impute import MissingIndicator

indicator = MissingIndicator(features="missing-only")

indicator_values = indicator.fit_transform(final_data)

indicator_columns = indicator.features_

print(indicator_columns)
```

Interpretation:

This cell uses scikit-learn's MissingIndicator to identify exactly which columns contain any missing values at all, printing just that list of 'missing-only' features.

STEP 41

Code:

```
import numpy as np

random_sample = final_data.copy()

for col in numerical_cols:
    # Only attempt random sampling if the column has at least one non-null value to
sample from
    if random_sample[col].notna().any():
        # Check if the column also has missing values to impute
        if random_sample[col].isnull().sum() > 0:

            random_values = random_sample[col].dropna().sample(
                random_sample[col].isnull().sum(),
                replace=True,
                random_state=42
            )

            random_values.index = random_sample[
                random_sample[col].isnull()
            ].index

            random_sample.loc[
                random_sample[col].isnull(),
                col
            ] = random_values
```

Interpretation:

This cell manually performs random sample imputation: for each numerical column with missing values, it randomly draws replacement values from the column's own existing (non-null) values and places them into the missing positions, preserving the original distribution of the data.

STEP 42

Code:

```
random_sample.isnull().sum()
```

Interpretation:

This cell checks the missing-value counts again after random sample imputation, confirming that the previously missing numerical entries have now been filled.

STEP 43

Code:

```
from sklearn.impute import KNNImputer

knn_data = final_data.copy()

knn = KNNImputer(n_neighbors=5)

# Filter numerical columns to exclude those that are entirely NaN
imputable_numerical_cols = numerical_cols[knn_data[numerical_cols].notna().any()]

# Apply KNNImputer only to the imputable numerical columns
knn_data[imputable_numerical_cols] = knn.fit_transform(
    knn_data[imputable_numerical_cols]
)

knn_data.isnull().sum()
```

Interpretation:

This cell applies scikit-learn's KNNImputer (with 5 neighbours) to the numerical columns, filling missing values based on the values of the most similar rows rather than a single fixed statistic, then checks remaining nulls.

STEP 44

Code:

```
from sklearn.impute import KNNImputer

knn_data = final_data.copy()

knn = KNNImputer(n_neighbors=5)

# Filter numerical columns to exclude those that are entirely NaN
imputable_numerical_cols = numerical_cols[knn_data[numerical_cols].notna().any()]

# Apply KNNImputer only to the imputable numerical columns
knn_data[imputable_numerical_cols] = knn.fit_transform(
    knn_data[imputable_numerical_cols]
)

knn_data.isnull().sum()
```

Interpretation:

This cell repeats the KNN imputation step identically, effectively re-running the same 5-neighbour KNNImputer on the numerical columns as a confirmation or re-execution of the previous step.

STEP 45

Code:

```
from sklearn.experimental import enable_iterative_imputer

from sklearn.impute import IterativeImputer

mice_data = final_data.copy()

mice = IterativeImputer(random_state=42)

# Filter numerical columns to exclude those that are entirely NaN
imputable_numerical_cols = numerical_cols[mice_data[numerical_cols].notna().any()]

# Apply IterativeImputer only to the imputable numerical columns
mice_data[imputable_numerical_cols] = mice.fit_transform(
    mice_data[imputable_numerical_cols]
)

mice_data.isnull().sum()
```

Interpretation:

This cell applies scikit-learn's IterativeImputer (the MICE technique), which models each numerical column with missing values as a function of the other columns and iteratively refines its estimates, then checks how many nulls remain.

STEP 46

Code:

```
import requests
import pandas as pd

url = "https://dummyjson.com/users"

response = requests.get(url)

api_df = pd.DataFrame(response.json()["users"])

api_df.head()
```

Interpretation:

This cell re-fetches the same user data from the dummyjson.com API and rebuilds api_df, refreshing the API-based data source before it is reused later in the notebook.

STEP 47

Code:

```
api_df = api_df.iloc[:8].copy()

api_df["id"] = [101,102,103,104,105,106,107,108]
```

Interpretation:

This cell trims api_df down to its first eight rows and manually overwrites the id column with a fixed sequence of customer IDs (101-108), aligning the API data with the notebook's own customer_id numbering so the two can be joined cleanly.

STEP 48

Code:

```
final_data = pd.merge(
    merged_data,
    api_df,
    left_on="customer_id",
    right_on="id",
    how="left"
)

final_data.isnull().sum()
```

Interpretation:

This cell re-merges merged_data with the newly aligned api_df on customer_id/id and immediately checks the missing-value counts of the refreshed final_data, confirming how the join affected data completeness.

STEP 49

Code:

```
import numpy as np
import pandas as pd

from scipy.stats import zscore
from scipy.stats.mstats import winsorize
```

Interpretation:

This cell imports NumPy, Pandas, and two statistical helpers from SciPy — zscore for standardising values and winsorize for capping extreme values — in preparation for the outlier-detection techniques that follow.

STEP 50

Code:

```
numerical_cols = final_data.select_dtypes(include=["int64","float64"]).columns

print(numerical_cols)
```

Interpretation:

This cell re-selects the numerical columns of final_data and prints them, confirming which fields the upcoming outlier-detection methods will be applied to.

STEP 51

Code:

```
z_scores = np.abs(zscore(final_data[numerical_cols]))

z_scores = pd.DataFrame(
    z_scores,
    columns=numerical_cols
)

z_scores.head()
```

Interpretation:

This cell calculates the absolute Z-score for every value in the numerical columns, which measures how many standard deviations each value is from its column's mean, and stores the results in a DataFrame for inspection.

STEP 52

Code:

```
threshold = 3

zscore_data = final_data[
    (z_scores < threshold).all(axis=1)
]

print("Original Shape :", final_data.shape)

print("After Z-Score :", zscore_data.shape)
```

Interpretation:

This cell keeps only the rows where every numerical column has a Z-score below 3 (a common outlier threshold), producing zscore_data, and prints the row counts before and after to show how many records were removed.

STEP 53

Code:

```
iqr_data = final_data.copy()

for col in numerical_cols:

    Q1 = iqr_data[col].quantile(0.25)

    Q3 = iqr_data[col].quantile(0.75)
```

```
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR

    upper = Q3 + 1.5 * IQR

    iqr_data = iqr_data[
        (iqr_data[col] >= lower) &
        (iqr_data[col] <= upper)
    ]

print("Original Shape :", final_data.shape)

print("After IQR :", iqr_data.shape)
```

Interpretation:

This cell applies the IQR (Interquartile Range) method: for each numerical column it calculates the acceptable lower and upper bounds from the 25th and 75th percentiles, then repeatedly filters the data to keep only rows within those bounds, printing the shape before and after.

STEP 54

Code:

```
percentile_data = final_data.copy()

for col in numerical_cols:

    lower = percentile_data[col].quantile(0.01)

    upper = percentile_data[col].quantile(0.99)

    percentile_data = percentile_data[
        (percentile_data[col] >= lower) &
        (percentile_data[col] <= upper)
    ]

print("Original Shape :", final_data.shape)

print("After Percentile :", percentile_data.shape)
```

Interpretation:

This cell applies a percentile-based method, keeping only rows whose numerical values fall between the 1st and 99th percentile for every numeric column, and prints the resulting shape compared with the original.

STEP 55

Code:

```
winsor_data = final_data.copy()
```

```
for col in numerical_cols:

    winsor_data[col] = winsorize(
        winsor_data[col],
        limits=[0.05,0.05]winsor_data[numerical_cols].describe()
    )
```

Interpretation:

This cell applies Winsorization to each numerical column, which caps the most extreme 5% of values at both ends instead of removing the rows entirely, so outliers are pulled in towards the rest of the data rather than deleted.

STEP 56

Code:

```
winsor_data[numerical_cols].describe()
```

Interpretation:

This cell prints summary statistics for the numerical columns after Winsorization, allowing a check of how the minimum, maximum and other statistics changed once extreme values were capped.

STEP 57

Code:

```
import matplotlib.pyplot as plt
import seaborn as sns

for col in numerical_cols:

    plt.figure(figsize=(6,4))

    sns.boxplot(
        x=final_data[col],
        color="skyblue"
    )

    plt.title(f"Before Outlier Handling - {col}")

    plt.show()
```

Interpretation:

This cell draws a boxplot for every numerical column using the original final_data, giving a 'before' picture of outliers prior to any outlier treatment.

STEP 58

Code:

```
for col in numerical_cols:
```

```
    plt.figure(figsize=(6,4))

    sns.boxplot(
        x=winsor_data[col],
        color="lightgreen"
    )

    plt.title(f"After Winsorization - {col}")

    plt.show()
```

Interpretation:

This cell draws the same boxplots again but using the Winsorized data, giving an 'after' picture so the effect of Winsorization on each feature's spread can be compared directly with the 'before' plots.

STEP 59

Code:

```
comparison = pd.DataFrame({

    "Method":[
        "Original",
        "Z-Score",
        "IQR",
        "Percentile",
        "Winsorization"
    ],

    "Rows":[
        final_data.shape[0],
        zscore_data.shape[0],
        iqr_data.shape[0],
        percentile_data.shape[0],
        winsor_data.shape[0]
    ]

})

comparison
```

Interpretation:

This cell builds a small summary table comparing the number of rows that remain after each outlier-handling method (Original, Z-Score, IQR, Percentile, Winsorization), making it easy to see which method removed the most or fewest records.

STEP 60

Code:

```
final_data.dtypes
```

Interpretation:

This cell lists the data types of every column in final_data again, used here as a checkpoint before the date and mixed-variable columns are converted in the following steps.

STEP 61

Code:

```
date_columns = ["birthDate"]

for col in date_columns:
    final_data[col] = pd.to_datetime(final_data[col], errors="coerce")

final_data[date_columns].head()
```

Interpretation:

This cell converts the birthDate column from plain text into a proper datetime type using pd.to_datetime, with invalid values coerced to missing rather than raising an error, and previews the converted column.

STEP 62

Code:

```
final_data.info()
```

Interpretation:

This cell runs info() again to confirm that the birthDate column now has a proper datetime data type after the conversion in the previous step.

STEP 63

Code:

```
today = pd.Timestamp.today()

final_data["days_since_last_purchase"] = (
    today - final_data["date"]
).dt.days

final_data[["date", "days_since_last_purchase"]].head()
```

Interpretation:

This cell calculates how many days have passed between today's date and each transaction's date column, storing the result as a new feature called days_since_last_purchase, and previews the new column alongside the original date.

STEP 64

Code:

```
final_data["days_since_birth"] = (
    today - final_data["birthDate"]
).dt.days

final_data[["birthDate", "days_since_birth"]].head()
```

Interpretation:

This cell calculates the number of days between today and each customer's birthDate, creating a new days_since_birth feature that captures customer age in days rather than years.

STEP 65

Code:

```
final_data["birth_year"] = final_data["birthDate"].dt.year
final_data["birth_month"] = final_data["birthDate"].dt.month
final_data["birth_day"] = final_data["birthDate"].dt.day

final_data.head()
```

Interpretation:

This cell extracts the year, month and day components from the birthDate column and stores them as three new columns (birth_year, birth_month, birth_day), breaking the single date into separate, model-friendly parts.

STEP 66

Code:

```
final_data["birth_day_name"] = final_data["birthDate"].dt.day_name()

final_data[["birthDate", "birth_day_name"]].head()
```

Interpretation:

This cell extracts the name of the weekday (e.g. Monday, Tuesday) that each customer was born on from birthDate and stores it as a new birth_day_name column.

STEP 67

Code:

```
final_data["customer_id"] = (
    final_data["customer_id"]
    .astype(str)
    .str.extract("(\d+)")
    .astype(int)
)

final_data["customer_id"].head()
```

Interpretation:

This cell cleans the customer_id column by converting it to text, extracting only the numeric digits with a regular expression, and converting the result back to an integer — this handles cases where the ID field contains extra non-numeric characters.

STEP 68

Code:

```
final_data["product_id_numeric"] = (
    final_data["product_id"]
    .astype(str)
    .str.extract("(\d+)")
    .astype(int)
)

final_data[["product_id", "product_id_numeric"]].head()
```

Interpretation:

This cell applies the same digit-extraction cleaning to the product_id column, creating a new numeric-only column called product_id_numeric alongside the original mixed-format product ID.

STEP 69

Code:

```
final_data.head()
```

Interpretation:

This cell displays the first five rows of final_data again, checking that the date-handling and ID-cleaning steps produced the expected columns and values.

STEP 70

Code:

```
final_data.isnull().sum()
```

Interpretation:

This cell checks the missing-value counts across final_data once more, confirming the state of the dataset right before categorical encoding begins.

STEP 71

Code:

```
categorical_cols = final_data.select_dtypes(include="object").columns

print(categorical_cols)
```

Interpretation:

This cell selects the categorical (object-type) columns from final_data and prints their names, identifying exactly which fields will need to be encoded into numbers for machine learning.

STEP 72

Code:

```
from sklearn.preprocessing import LabelEncoder

label_data = final_data.copy()

label_encoder = LabelEncoder()

label_columns = ["gender_x"] # Changed 'gender' to 'gender_x'

for col in label_columns:
    label_data[col] = label_encoder.fit_transform(label_data[col])

label_data.head()
```

Interpretation:

This cell applies scikit-learn's LabelEncoder to the gender_x column, converting its text categories into numeric codes, and stores the result in a new DataFrame called label_data.

STEP 73

Code:

```
onehot_data = pd.get_dummies(
    label_data,
    columns=["city", "payment_mode"],
    drop_first=True
)

onehot_data.head()
```

Interpretation:

This cell applies one-hot encoding to the city and payment_mode columns using pd.get_dummies (dropping the first category of each to avoid redundancy), turning each category into its own 0/1 column, and stores the result as onehot_data.

STEP 74

Code:

```
from sklearn.preprocessing import OrdinalEncoder

ordinal_data = onehot_data.copy()

education_order = [
    ["High School",
     "Diploma",
     "Bachelor",
```

```
     "Master",
     "PhD"]
]

ordinal_encoder = OrdinalEncoder(categories=education_order)

ordinal_data[["education"]] = ordinal_encoder.fit_transform(
    ordinal_data[["education"]]
)

ordinal_data.head()
```

Interpretation:

This cell applies OrdinalEncoder to the education column using a manually specified order (High School to PhD), converting education level into a single number that respects the natural ranking of the categories.

STEP 75

Code:

```
print(ordinal_data.columns)
```

Interpretation:

This cell prints the full column list of ordinal_data, giving a checkpoint view of every original and newly-encoded feature at this stage of the pipeline.

STEP 76

Code:

```
ordinal_data["income_group"] = pd.cut(

    ordinal_data["income"],

    bins=[0,30000,60000,90000,120000,np.inf],

    labels=[
        "Low",
        "Medium",
        "High",
        "Very High",
        "Premium"
    ]

)

ordinal_data[["income","income_group"]].head()
```

Interpretation:

This cell creates an income_group feature by cutting the income column into five labelled bands (Low, Medium, High, Very High, Premium) using fixed bin edges, turning a continuous income value into a readable category.

STEP 77

Code:

```
income_encoder = LabelEncoder()

ordinal_data["income_group"] = income_encoder.fit_transform(
    ordinal_data["income_group"]
)

ordinal_data.head()
```

Interpretation:

This cell converts the newly created income_group category into numeric codes using LabelEncoder, so the income band can be used directly as a numeric feature in later modelling.

STEP 78

Code:

```
ordinal_data.dtypes
```

Interpretation:

This cell lists the data types of every column in ordinal_data, confirming that the categorical fields have now been successfully converted into numeric form.

STEP 79

Code:

```
ordinal_data.head()
```

Interpretation:

This cell previews the first five rows of ordinal_data, giving a final look at the dataset after all the encoding steps have been applied.

STEP 80

Code:

```
from sklearn.preprocessing import (
    StandardScaler,
    MinMaxScaler,
    MaxAbsScaler,
    RobustScaler,
    Normalizer
)

from sklearn.compose import ColumnTransformer
```

Interpretation:

This cell imports several scikit-learn scaling tools — StandardScaler, MinMaxScaler, MaxAbsScaler, RobustScaler and Normalizer — plus ColumnTransformer, in preparation for comparing different feature scaling techniques.

STEP 81

Code:

```
numerical_cols = ordinal_data.select_dtypes(
    include=["int64","float64"]
).columns

print(numerical_cols)
```

Interpretation:

This cell re-selects the numerical columns from ordinal_data and prints them, identifying exactly which fields the upcoming scaling techniques will be applied to.

STEP 82

Code:

```
standard_data = ordinal_data.copy()

standard_scaler = StandardScaler()

standard_data[numerical_cols] = standard_scaler.fit_transform(
    standard_data[numerical_cols]
)

standard_data.head()
```

Interpretation:

This cell applies StandardScaler to the numerical columns, transforming each one to have a mean of 0 and a standard deviation of 1, and previews the scaled result as standard_data.

STEP 83

Code:

```
minmax_data = ordinal_data.copy()

minmax_scaler = MinMaxScaler()

minmax_data[numerical_cols] = minmax_scaler.fit_transform(
    minmax_data[numerical_cols]
)

minmax_data.head()
```

Interpretation:

This cell applies MinMaxScaler to the numerical columns, rescaling every value into a fixed 0-to-1 range, and stores/previews the result as minmax_data.

STEP 84

Code:

```
maxabs_data = ordinal_data.copy()

maxabs_scaler = MaxAbsScaler()

maxabs_data[numerical_cols] = maxabs_scaler.fit_transform(
    maxabs_data[numerical_cols]
)

maxabs_data.head()
```

Interpretation:

This cell applies MaxAbsScaler to the numerical columns, scaling each value by the maximum absolute value in its column so that everything falls between -1 and 1 while preserving sparsity, and previews the result as maxabs_data.

STEP 85

Code:

```
robust_data = ordinal_data.copy()

robust_scaler = RobustScaler()

robust_data[numerical_cols] = robust_scaler.fit_transform(
    robust_data[numerical_cols]
)

robust_data.head()
```

Interpretation:

This cell applies RobustScaler to the numerical columns, which scales data using the median and interquartile range instead of the mean and standard deviation, making it less sensitive to outliers, and previews the result as robust_data.

STEP 86

Code:

```
normalizer_data = ordinal_data.copy()

normalizer = Normalizer()

normalizer_data[numerical_cols] = normalizer.fit_transform(
    normalizer_data[numerical_cols]
)
```

```
normalizer_data.head()
```

Interpretation:

This cell applies a Normalizer to the numerical columns, rescaling each row so that its values form a unit vector (useful when the direction/proportion between features matters more than their absolute size), and previews the result.

STEP 87

Code:

```
column_transformer = ColumnTransformer(

    transformers=[

        (
            "standard",
            StandardScaler(),
            ["income","amount"]
        ),

        (
            "minmax",
            MinMaxScaler(),
            ["age_x"] # Changed 'age' to 'age_x'
        )

        # Removed RobustScaler for 'total_purchases' as it does not exist
    ],

    remainder="passthrough"

)
```

Interpretation:

This cell sets up a ColumnTransformer that applies StandardScaler to the income and amount columns and MinMaxScaler to the age_x column in a single, organised transformation step, leaving all other columns untouched ('passthrough').

STEP 88

Code:

```
scaled_data = pd.DataFrame(scaled_array)

scaled_data.head()
```

Interpretation:

This cell converts the array produced by the ColumnTransformer back into a readable DataFrame called scaled_data and previews its first rows.

STEP 89

Code:

```
print("Original Shape :", ordinal_data.shape)

print("Scaled Shape :", scaled_data.shape)
```

Interpretation:

This cell prints the shape of the dataset before and after the ColumnTransformer step, confirming that scaling changed the values but not the number of rows or the overall structure.

STEP 90

Code:

```
import numpy as np

from sklearn.preprocessing import (
    FunctionTransformer,
    PowerTransformer,
    Binarizer
)
```

Interpretation:

This cell imports FunctionTransformer, PowerTransformer and Binarizer from scikit-learn along with NumPy, setting up the tools needed for the mathematical feature transformations that follow.

STEP 91

Code:

```
feature_data = ordinal_data.copy()

feature_data["purchase_per_day"] = (
    feature_data["amount"] / # Replaced 'total_purchases' with 'amount'
    (feature_data["days_since_last_purchase"] + 1) # Replaced 'days_since_signup' with
'days_since_last_purchase'
)

feature_data[
    ["amount", # Changed 'total_purchases' to 'amount'
     "days_since_last_purchase", # Changed 'days_since_signup' to
'days_since_last_purchase'
     "purchase_per_day"]
].head()
```

Interpretation:

This cell creates a new feature called purchase_per_day by dividing each transaction's amount by the number of days since that customer's last purchase (plus one, to avoid dividing by zero), and previews the new ratio feature.

STEP 92

Code:

```
customer_transaction_counts =
feature_data.groupby('customer_id')['transaction_id'].transform('count')
feature_data["total_purchases"] = customer_transaction_counts

feature_data["average_purchase_amount"] = (
    feature_data["amount"] /
    (feature_data["total_purchases"] + 1)
)

feature_data[
    ["amount",
     "total_purchases",
     "average_purchase_amount"]
].head()
```

Interpretation:

This cell counts how many transactions each customer has made, stores that count as total_purchases, and then creates an average_purchase_amount feature by dividing amount by the customer's total purchase count.

STEP 93

Code:

```
log_transformer = FunctionTransformer(
    np.log1p,
    validate=False
)

feature_data["income_log"] = log_transformer.transform(
    feature_data[["income"]]
)
```

Interpretation:

This cell applies a log transformation (log1p) to the income column using FunctionTransformer, creating an income_log feature that compresses large income values and reduces skewness.

STEP 94

Code:

```

sqrt_transformer = FunctionTransformer(
    np.sqrt,
    validate=False
)

feature_data["income_sqrt"] = sqrt_transformer.transform(
    feature_data[["income"]]
)
```

Interpretation:

This cell applies a square-root transformation to the income column, creating an income_sqrt feature as another way of reducing the impact of very large income values.

STEP 95

Code:

```
reciprocal_transformer = FunctionTransformer(
    lambda x: 1/(x+1),
    validate=False
)

feature_data["income_reciprocal"] = reciprocal_transformer.transform(
    feature_data[["income"]]
)
```

Interpretation:

This cell applies a reciprocal transformation (1 divided by value+1) to the income column, creating an income_reciprocal feature that inverts the scale of income values.

STEP 96

Code:

```
boxcox = PowerTransformer(method="box-cox")

feature_data["income_boxcox"] = boxcox.fit_transform(
    feature_data[["income"]] + 1
)
```

Interpretation:

This cell applies a Box-Cox power transformation to the income column (shifted by 1 to keep all values positive, since Box-Cox requires strictly positive data), creating an income_boxcox feature that reshapes the distribution to be more normal.

STEP 97

Code:

```
yeojohnson = PowerTransformer(method="yeo-johnson")

feature_data["income_yeojohnson"] = yeojohnson.fit_transform(
    feature_data[["income"]]
)
```

Interpretation:

This cell applies a Yeo-Johnson power transformation to the income column, which works similarly to Box-Cox but also supports zero and negative values, creating an income_yeojohnson feature.

STEP 98

Code:

```
feature_data["income_group"] = pd.cut(

    feature_data["income"],

    bins=4,

    labels=[
        "Low",
        "Medium",
        "High",
        "Very High"
    ]

)

feature_data[
    ["income",
     "income_group"]
].head()
```

Interpretation:

This cell bins the income column into four equal-width groups labelled Low, Medium, High and Very High, creating a categorical income_group feature and previewing it alongside the original income values.

STEP 99

Code:

```
feature_data["frequent_buyer"] = np.where(
    feature_data["total_purchases"] > 5,
    1,
    0
)

feature_data[
    ["total_purchases",
     "frequent_buyer"]
].head()
```

Interpretation:

This cell creates a binary frequent_buyer flag that is set to 1 when a customer's total_purchases is greater than 5, and 0 otherwise, turning purchase count into a simple yes/no feature.

STEP 100

Code:

```
binarizer = Binarizer(threshold=50000)

feature_data["income_binary"] = binarizer.fit_transform(
    feature_data[["income"]]
```

```
)

feature_data[
    ["income",
     "income_binary"]
].head()
```

Interpretation:

This cell applies a Binarizer with a threshold of 50,000 to the income column, creating an income_binary feature that is 1 for incomes above the threshold and 0 otherwise.

STEP 101

Code:

```
feature_data.head()
```

Interpretation:

This cell displays the first five rows of feature_data, giving a full view of the original fields together with every engineered feature created in the steps above.

STEP 102

Code:

```
print("Dataset Shape :", feature_data.shape)
```

Interpretation:

This cell prints the final shape of feature_data, confirming the total number of rows and columns in the fully processed and feature-engineered dataset.

STEP 103

Code:

```
feature_data.to_csv(
    "processed_customer_data.csv",
    index=False
)

print("Dataset Exported Successfully!")
```

Interpretation:

This cell exports feature_data to a CSV file named processed_customer_data.csv and prints a confirmation message, saving the finished dataset so it can be reused for model training without repeating the whole pipeline.

STEP 104

Code:

```
import joblib

joblib.dump(column_transformer, "feature_pipeline.pkl")

print("Pipeline Saved Successfully!")
```

Interpretation:

This cell uses joblib to save the fitted column_transformer object to a file called feature_pipeline.pkl, so the exact same scaling steps can be reloaded and reapplied to new data later without retraining.

STEP 105

Code:

```
processed_data = pd.read_csv("processed_customer_data.csv")

processed_data.head()
```

Interpretation:

This cell reads the exported processed_customer_data.csv back into a new DataFrame called processed_data and previews it, verifying that the file was saved correctly and can be loaded again from disk.

STEP 106

Markdown Content:

Project Conclusion

This project successfully developed a complete Data Cleaning, Data Preprocessing, and Feature Engineering Pipeline for customer purchase propensity analysis. Data was collected from multiple sources, including a CSV file, JSON file, SQL database, and REST API, and merged into a single dataset. The dataset was explored using Exploratory Data Analysis (EDA) to understand its structure, distributions, and relationships. Missing values were handled using various imputation techniques, outliers were detected and treated, categorical variables were encoded, numerical features were scaled, and new meaningful features were engineered. The final processed dataset was exported as processed_customer_data.csv, making it clean, consistent, and ready for future Machine Learning models that predict customer purchase behavior.

Interpretation:

This markdown cell documents the reasoning, explanation, or concluding observations associated with the notebook at this point in the workflow.

STEP 107

Markdown Content:

Project Conclusion

This project successfully developed a complete Data Cleaning, Data Preprocessing, and Feature Engineering Pipeline for customer purchase propensity analysis. Data was collected from multiple

sources, including a CSV file, JSON file, SQL database, and REST API, and merged into a single dataset. The dataset was explored using Exploratory Data Analysis (EDA) to understand its structure, distributions, and relationships. Missing values were handled using various imputation techniques, outliers were detected and treated, categorical variables were encoded, numerical features were scaled, and new meaningful features were engineered. The final processed dataset was exported as processed_customer_data.csv, making it clean, consistent, and ready for future Machine Learning models that predict customer purchase behavior.

Interpretation:

This markdown cell documents the reasoning, explanation, or concluding observations associated with the notebook at this point in the workflow.

STEP 108

Markdown Content:

Important Techniques Used in the Project

Imported data from CSV, JSON, SQL Database, and REST API. Merged datasets using common keys such as customer_id and product_id. Performed Exploratory Data Analysis (EDA) using histograms, boxplots, countplots, heatmaps, pairplots, and descriptive statistics. Handled missing values using: Simple Imputer Most Frequent Imputation Random Sample Imputation KNN Imputer MICE (Iterative Imputer) Complete Case Analysis Detected and handled outliers using: Z-Score Method IQR Method Percentile Method Winsorization Processed date and mixed variables by converting dates into datetime format and creating derived features. Applied categorical encoding techniques: Label Encoding One-Hot Encoding Ordinal Encoding Income Binning Scaled numerical features using: StandardScaler MinMaxScaler MaxAbsScaler RobustScaler Normalizer ColumnTransformer Performed feature engineering by creating new interaction features, applying transformations (Log, Square Root, Reciprocal, Box-Cox, Yeo-Johnson), binning, and binarization. Exported the final processed dataset for Machine Learning applications.

Interpretation:

This markdown cell documents the reasoning, explanation, or concluding observations associated with the notebook at this point in the workflow.

STEP 109

Markdown Content:

Future Scope

This project can be extended in several ways:

Train Machine Learning models such as Logistic Regression, Decision Tree, Random Forest, XGBoost, or Neural Networks to predict customer purchases. Develop a real-time customer purchase prediction system integrated with an e-commerce website. Build interactive dashboards using Power BI or Tableau for business decision-making. Automate the preprocessing pipeline using Scikit-learn Pipelines for production environments. Integrate live APIs and cloud databases to process real-time customer and transaction data. Use advanced feature engineering and hyperparameter tuning to improve prediction accuracy. Deploy the trained model as a web application or REST API using Flask, FastAPI, or Django.

Interpretation:

This markdown cell documents the reasoning, explanation, or concluding observations associated with the notebook at this point in the workflow.

STEP 110

Markdown Content:

Final Outcome:

The project successfully transformed raw customer, transaction, product, and API data into a clean, structured, and feature-engineered dataset. The preprocessing pipeline improved data quality, reduced inconsistencies, and prepared the dataset for predictive analytics. This workflow demonstrates industry-standard data preprocessing practices and provides a strong foundation for building accurate customer purchase prediction models in real-world e-commerce applications.

Interpretation:

This markdown cell documents the reasoning, explanation, or concluding observations associated with the notebook at this point in the workflow.

Project Summary

The notebook implements an end-to-end data cleaning, preprocessing, and feature engineering pipeline for a customer purchase dataset. Data was collected from four different sources (CSV, JSON, SQL database, and a REST API) and merged into a single table. The pipeline then performs exploratory data analysis, compares multiple missing-value imputation techniques (mean, median, most-frequent, constant, random sample, KNN, and MICE), detects and treats outliers using four different methods (Z-score, IQR, percentile, and Winsorization), engineers date-based and ID-based features, encodes categorical variables (label, one-hot, and ordinal encoding), scales numerical features using five different scalers, and finally engineers new interaction and transformed features (ratios, log/sqrt/reciprocal/Box-Cox/Yeo-Johnson transforms, binning, and binarization) before exporting the finished dataset to processed_customer_data.csv.